

D05. Linux and Bash Operations Run Instructions

Overview

This deliverable documents the Linux and Bash operational layer used to execute the selected FlowCore scenario. The Bash scripts support one single simplified inbound warehouse scenario and provide a repeatable execution sequence for preparing the environment, running the scenario, validating the result, and clearing generated outputs.

Project Files Used by the Bash Layer

The Linux and Bash layer works with the following folders:

- **config/**
- **scripts/**
- **runtime/logs/**
- **runtime/snapshots/**

The current Bash layer uses these input files:

- **config/locations.csv**
- **config/receipt_data.csv**
- **config/assignment_rules.conf**

The current operational scripts are:

- **scripts/setup_demo.sh**
- **scripts/run_demo.sh**
- **scripts/validate_demo.sh**
- **scripts/reset_demo.sh**

Run Sequence

The standard execution sequence of the demo is:

1. **Prepare the runtime environment**
2. **Execute the inbound scenario**
3. **Validate the generated result**
4. **Clear generated outputs if another clean run is required**

The commands are executed from the project root in the Linux or WSL environment.

Before the first run, execution permission is assigned to the four Bash scripts. The normal execution sequence is then setup, run, and validate.

The normal command sequence is:

```
cd /mnt/c/Users/<username>/Desktop/FlowCore_Master/project
```

```
chmod u+x scripts
```

```
scripts/setup_demo.sh && scripts/run_demo.sh && scripts/validate_demo.sh
```

The `chmod u+x` command is normally required only once, unless file permissions are changed again later, or other users should also be able to execute this file. After a successful run, the project is expected to generate the runtime log and snapshot files and the validation script should report a successful result. If another clean run is required, the runtime outputs can be cleared with:

```
scripts/reset_demo.sh
```

Script Overview

Setup_demo.sh

This script prepares the runtime environment.

Its purpose is to:

- **create the runtime folders if they do not exist**
- **clear previous generated files from earlier runs**
- **confirm that the runtime area is ready**

Run_demo.sh

This script performs the main execution of the selected inbound scenario.

Its purpose is to:

- **read the current configuration files**
- **process the inbound pallet records**
- **assign one valid location to each pallet**
- **record status and event changes**
- **generate final runtime snapshots**

The current execution logic uses the configured preferred zone, ignores inactive locations unless explicitly allowed, and processes pallets up to the configured maximum run size.

Validate_demo.sh

This script checks whether the run produced the expected result.

Its purpose is to:

- confirm that the output files exist
- confirm that the expected number of pallets was processed
- confirm that the expected number of events was generated
- confirm that all pallets reached the final status
- confirm that no pallet remained unassigned
- confirm that no location capacity rule was violated

Reset_demo.sh

This script clears generated runtime outputs after a run.

Its purpose is to:

- clean the log files
- clean the snapshot files
- restore the runtime area to a reusable state

Input Files

Locations.csv

This file defines the available warehouse locations used by the inbound scenario.

It includes:

- location identifier
- warehouse zone
- capacity
- current used value
- active or inactive indicator
- simple priority value

Receipt_data.csv

This file defines the inbound receipt and pallet data used in the scenario.

It includes:

- receipt identifier
- dock identifier
- pallet identifier
- SKU
- quantity

Assignment_rules.conf

This file defines the simple runtime rules used by the Bash layer.
It includes:

- **preferred warehouse zone**
- **inactive-location allowance flag**
- **maximum pallets per run**
- **final expected status**

Output Files

Status_log.csv

This file is the main operational log produced by the run.
It records the event and status path of each processed pallet.
The current implementation records:

- RECEIVED
- ASSIGNED
- STORED

Pallet_state.csv

This file records the final state of each processed pallet.
It includes:

- **pallet identifier**
- **receipt identifier**
- **SKU**
- **quantity**
- **assigned location**
- **final status**

Location_usage.csv

This file records the location usage state after the run.
It includes:

- **location identifier**
- **zone**
- **capacity**
- **used value after execution**
- **priority**

Expected Validation Result

With the current project configuration, the expected result of the demo is:

- **3 pallets are processed**
- **all 3 pallets reach the final STORED state**
- **9 events are written to the status log**
- **no pallet remains unassigned**
- **no location exceeds its configured capacity**

In the current run, the three pallets are assigned to the following locations:

- **A01**
- **A02**
- **A03**